

01-编译器概览与词法分析复习讲义

目录

1	01 编译器概览与词法分析复习讲义
---	-------------------

2

1 01 编译器概览与词法分析复习讲义

对应资料：

- 讲解/Ch1-Introduction-to-Compilers.tex
- 讲解/Ch3-1-Finite-Automata.tex
- Slides/Slides/Ch1-Introduction to Compilers.pdf
- Slides/Slides/Ch3-1-Finite Automata.pdf
- Slides/Slides/Ch3-2-Lexical Analysis.pdf
- Review/compile.pdf
- Exam/2025.pdf 词法分析题
- Exam/过时/2022-exam-A.pdf 题目 1
- Exam/过时/2023-exam-A.pdf 题目 1

1.1 一、编译器总流程

编译器把源语言程序翻译为目标语言程序。典型阶段：

- 词法分析：字符流到 token 流。
- 语法分析：token 流到 AST 或 parse tree。
- 语义分析：类型、作用域、声明使用关系检查。
- 中间代码生成：AST 到 IR。
- 机器无关优化：在 IR 上做数据流分析、SSA、公共子表达式等。
- 目标代码生成：IR 到机器代码。
- 机器相关优化：寄存器分配、指令调度、窥孔优化。

答题时常写成：

```
Source Code -> Lexer -> Tokens -> Parser -> AST
              -> Semantic Analysis -> IR -> Optimization
              -> Target Code Generation -> Machine Code
```

1.2 二、字符串与语言操作

字符串操作：

ab : a 和 b 连接
 s^R : s 反转
 $|s|$: s 的长度
 ϵ : 空串, 长度为 0

语言操作：

$L_1 L_2 = \{ab \mid a \in L_1, b \in L_2\}$
 $L^0 = \{\epsilon\}$
 $L^n = \{s_1 s_2 \dots s_n \mid s_i \in L\}$
 $L^* = \bigcup_{i \geq 0} L^i$
 $L^+ = \bigcup_{i \geq 1} L^i$
 $\bar{L} = \{s \in \Sigma^* \mid s \notin L\}$
 $L^R = \{s^R \mid s \in L\}$

易错点：

- $\{\epsilon\}$ 不是空集。
- L^+ 不包含空串, 除非 $\epsilon \in L$ 。
- 正则语言对并、交、连接、反转、补、Kleene star 封闭。

1.3 三、正则表达式

正则表达式语义：

$\text{emptyset} : L(\text{emptyset}) = \text{emptyset}$
 $\epsilon : L(\epsilon) = \{\epsilon\}$
 $A \mid B : L(A \mid B) = L(A) \cup L(B)$
 $AB : L(AB) = L(A)L(B)$
 $A^* : L(A^*) = L(A)^*$
 $(A) : L((A)) = L(A)$

考试中正则到自动机固定考 Thompson 构造：

- ϵ : 一条 ϵ 边。
- 字符 a : 一条标记为 a 的边。
- $A \mid B$: 新起点 ϵ 到 A 、 B 起点; A 、 B 终点 ϵ 到新终点。
- AB : A 终点 ϵ 到 B 起点。
- A^* : 新起点 ϵ 到 A 起点和新终点; A 终点 ϵ 到 A 起点和新终点。

1.4 四、DFA

DFA 定义:

```

M = (Q, Sigma, delta, q0, F)
Q      : 状态集合
Sigma  : 字母表
delta  : Q x Sigma -> Q
q0     : 起始状态
F      : 终止状态集合

```

扩展转移:

```

delta*(q, epsilon) = q
delta*(q, xa) = delta(delta*(q, x), a)
L(M) = {s in Sigma* | delta*(q0, s) in F}

```

DFA 答题要点:

- 每个状态对每个输入字符有唯一后继。
- 接受条件是读完整个串后停在终止状态。
- 画图时起始状态有入箭头，终止状态双圈。

1.5 五、NFA

NFA 定义:

```

M = (Q, Sigma, delta, q0, F)
delta : Q x (Sigma union {epsilon}) -> 2^Q

```

接受条件:

```

L(M) = {s in Sigma* | delta*(q0, s) cap F != emptyset}

```

NFA 与 DFA 的区别:

- NFA 一个状态读一个字符可以到达多个状态。
- NFA 可以有 epsilon 转移。
- NFA 只要存在一条接受路径即可接受。
- DFA 与 NFA 识别能力相同，都识别正则语言。

1.6 六、NFA 到 DFA：子集构造法

输入：NFA $M = (Q, \Sigma, \delta, q_0, F)$ 。

输出：DFA，每个 DFA 状态是 NFA 状态的一个集合。

步骤：

1. 起始状态为 $\text{epsilon-closure}(\{q_0\})$ 。
2. 对每个 DFA 状态 S 和输入字符 a ，计算：

```
move(S, a) = union delta(q, a), q in S
next(S, a) = epsilon-closure(move(S, a))
```

1. 新集合还没出现就加入 DFA 状态集合。
2. 含有 NFA 终止状态的集合就是 DFA 终止状态。
3. 重复直到没有新状态。

考场表格写法：

DFA 状态	NFA 状态集合	on a	on b
A	{0,1,3}	B	C
B	{...}	...	...

1.7 七、DFA 最小化

目标：合并 ω -等价状态。

步骤：

1. 初始划分：终止状态一组，非终止状态一组。
2. 对每一组按转移目标所在分组继续拆分。
3. 重复拆分直到划分不再变化。
4. 每个等价类合成一个状态。

判断依据：

q_1 与 q_2 在同一组，当且仅当对每个输入字符 a ， $\delta(q_1, a)$ 与 $\delta(q_2, a)$ 仍落在同一组。

1.8 八、正则语言泵引理

泵引理形式：

若 L 是正则语言，则存在常数 C ，
对任意 $s \in L$ 且 $|s| \geq C$ ，
存在分解 $s = xyz$ ，满足 $|xy| \leq C$ ， $|y| > 0$ ，
并且对任意 $i \geq 0$ ， $xy^i z \in L$ 。

证明非正则语言的答题步骤：

1. 假设语言 L 是正则语言。
2. 取泵长度 C 。
3. 选一个长度不少于 C 的字符串 $s \in L$ 。
4. 对任意满足条件的分解 $s = xyz$ ，分析 y 的位置。
5. 取 $i=0$ 或 $i=2$ ，推出 $xy^i z \notin L$ 。
6. 与泵引理矛盾，故 L 不是正则语言。

1.9 九、自测题与答案

1.9.1 题 1：说明 $\{\epsilon\}$ 与 emptyset 的区别。

答案： emptyset 没有任何字符串， $\{\epsilon\}$ 有一个字符串，即空串。

1.9.2 题 2：DFA 与 NFA 识别能力是否相同？

答案：相同。NFA 可用子集构造法转换为等价 DFA，二者都识别正则语言。

1.9.3 题 3：子集构造法中 DFA 的终止状态如何确定？

答案：DFA 状态是 NFA 状态集合。只要该集合与 NFA 的终止状态集合相交，该 DFA 状态就是终止状态。

1.9.4 题 4：DFA 最小化第一步如何分组？

答案：按是否为终止状态分成两组：终止状态组和非终止状态组。